

Pt 2.1 Introduction to LLM Applications with LangChain

Instructions

This homework may be completed **individually or in pairs**. All required tasks must be completed. Optional tasks provide additional depth for motivated students.

Objectives

- Introduction to theoretical foundations of LLM-based applications.
- Understanding the basic workflow patterns and their application in real-world scenarios.
- Implementation of simple chatbots and LLM applications using basic workflow patterns.

Part I. Theoretical Foundations

1. Non-Reasoning vs. Reasoning Models

Research and explain the conceptual differences between **non-reasoning** and **reasoning** LLMs. Include:

- The main distinctions
- Typical use cases
- Implications for design of LLM-based applications

2. Prompting Techniques for Non-Agentic Applications

Conduct research on fundamental prompting techniques for applications that do not employ agent-based architectures. You can use the following resource as a first reading (basics section, from Introduction to Prompt Engineering)

https://learnprompting.org/docs/basics/prompt_engineering

Write (and understand) a short explanation covering:

1. **Basic prompting strategies** relevant for non-agentic applications (e.g., instruction prompts, zero-shot, few-shot, format-constrained prompts).
2. **The purpose of these strategies** Explain briefly what each technique tries to achieve.

3. **Recommendations** General best practices for prompt design.

Part II. Development Tasks

3. Console-Based Chatbot with Memory

Develop a simple console-based chatbot using **LangChain**.

Requirements:

- Implement memory using a basic structure (e.g., a Python list)
- Support multi-turn conversation
- Ensure the code is clear, functional, and minimally documented

4. Workflow Patterns

Read the article (only the introduction and the three workflow patterns):

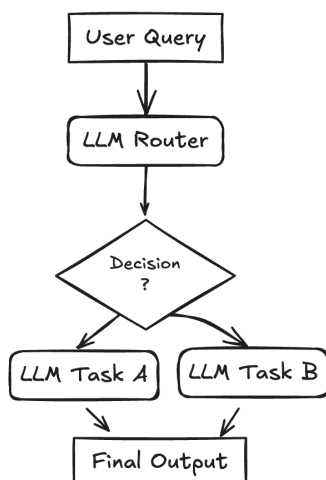
<https://www.philschmid.de/agent-c-pattern>

The article presents **three basic workflow patterns** for LLM applications.

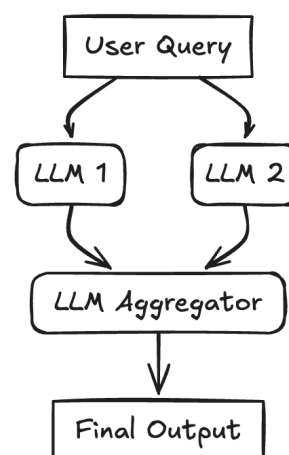
Workflow: Prompt Chaining



Workflow: Routing



Workflow: Parallelization



In this exercise you must adapt all three workflow pattern examples from the article to **LangChain**.

Requirements:

- Reimplement **each pattern** using LangChain `init_chat_model`
- Preserve the intended behavior and logic
- Verify that the code runs correctly
- You do not need to deliver these implementations, but you will need them later for exercise 5

5. Mini Applications Based on Workflow Patterns

Choose at least two workflow patterns from Exercise 4, build a small LLM application for each pattern using LangChain.

Requirements:

- Do **not** reuse scenarios from the article
- Each mini app must clearly illustrate the selected pattern
- Provide a brief explanation of your design choices and how the pattern is applied

Additional Requirement: Token Usage Tracking

Understanding and tracking token consumption is an essential practice when developing real-world LLM applications. Different models and providers apply pricing based on input, output, and reasoning tokens; therefore, monitoring token usage helps developers make informed decisions about cost optimization, model selection, prompt design, and system scalability.

For **each mini application**, implement a simple mechanism to track token usage throughout the entire pipeline. Tracking must include:

- **input_tokens**
- **output_tokens**
- **reasoning_tokens** (if available from the model or provider)

You may implement this with any basic structure (e.g., a Python dictionary, dataclass, or a simple object). At the end of each user interaction or pipeline execution, display the total tokens used.

6. Combined Multi-Pattern Application with Streamlit

Develop a complete LLM application integrating at least **two** workflow patterns using **LangChain** and **Streamlit**.

Requirements:

- Include a simple memory mechanism
- First, implement a simple console app
- Implement a simple streamlit app, just a single file. We will move to better architectures later in this course
- Document design choices and explain how each pattern is integrated

Additional Requirement: Token Usage Tracking

Understanding and tracking token consumption is an essential practice when developing real-world LLM applications. Different models and providers apply pricing based on input, output, and reasoning tokens; therefore, monitoring token usage helps developers make informed decisions about cost optimization, model selection, prompt design, and system scalability.

The application must include a mechanism to track **global token usage** across the entire user session or pipeline. Track and display at least:

- **input_tokens**
- **output_tokens**
- **reasoning_tokens** (if available from the model or provider)

You may implement this with any basic structure (e.g., a Python dictionary, dataclass, or a simple object). At the end of each user interaction or pipeline execution, display the total tokens used.

7. Research on LLM Evaluation Techniques (optional)

Conduct a brief investigation into evaluation techniques for LLM-based applications.

Recommended aspects:

- Automated evaluation methods
- Human-driven evaluation strategies
- Strengths and weaknesses of each
- Appropriate use cases for each method

Deliverables

1. A written report including all theoretical explanations and design justifications (see description below).
2. Source code for all implementations (pycharm project).
 - exercise 3:
 - chatbot.py
 - exercise 5:
 - workflow_patterns_chains.py
 - workflow_patterns_router.py
 - workflow_patterns_parallelization.py
 - exercise 6:
 - app.py
 - gui_app.py

Final Report

Sections:

1. Identification of Team Members

- Full names of all members (individual or pair).
- Brief description of how the work was distributed.

2. Responses to Theoretical Questions

- Complete and clearly written answers for all exercises in Part I.

3. Implementation Documentation. For each exercise in Part II, include:

- A brief description of the developed application.
- Design decisions made during implementation and justification for each decision.
- Difficulties encountered and how they were resolved.
- Description of each member's contributions.

4. Token Usage Discussion (where required)

- Explain the token-tracking mechanism implemented.
- Summarize the results and insights drawn from tracking usage.

5. References

- List all external sources used (articles, documentation, tutorials, etc.).

Grading

- Exercises 1-2 (theory): 10%
- Exercise 3 (simple console chabot): 10%
- Exercise 5 (workflow patterns mini apps): 20%
- Exercise 6 (final app): 50%
- Final Report: 10%